String Basics & Manipulations

Slicing a String

Given the string `text = 'HelloWorld'`, extract `'Hello'` using slicing.

In [1]:
```
text = 'HelloWorld'
print(text[0:5])
```

```
Hello
```

Negative Index in String

Given the string `text = 'HelloWorld'`, extract `'World'` using negative indices only.

In [2]:
```
print(text[5:10])
```

```
World
```

Step in Slicing

Given the string `text = 'abcdefgh'`, slice out every second character to get `'aceg'`.

In [3]:
```
text = 'abcdefgh'
print(text[0:7:2])
```

```
aceg
```

Count Method

Given the string `sentence = 'banana'`, use a string function to find how many times `'a'` appears.

In [4]:
```
sentence = 'banana'
print(sentence.count('a'))
```

```
3
```

Replace Method

Given the string `greeting = 'Hello, John!'`, replace `'John'` with `'Alice'`.

In [5]:
```
greeting = 'Hello, John!'
greeting.replace('John', 'Alice')
```

Out[5]:
```
'Hello, Alice!'
```

Upper & Lower

Convert `'PyThOn'` to all uppercase and all lowercase, then print both forms.

In [6]:
```
text = 'PyThOn'
print(text.upper())
print(text.lower())
```

```
PYTHON
python
```

Startswith/Endswith

Check if the string `url = 'https://example.com'` starts with `'https'` and ends with `'.com'`.

In [7]:
```
url = 'https://example.com'
print(url.startswith('https://'))
print(url.endswith('.com'))
```

```
True
True
```

Strip Method

Given `user_input = ' spaced out '` , remove the leading and trailing whitespace.

In [8]:
```
1 user_input = ' spaced out '
2 print(user_input.strip())
```

```
spaced out
```

Find (or Index) Method

Given `paragraph = 'The quick brown fox jumps over the lazy dog'` , find the index where `'fox'` occurs.

In [9]:
```
1 paragraph = 'The quick brown fox jumps over the lazy dog.'
2 print(paragraph.find('fox'))
```

```
16
```

String Split & Join

Given `text = 'One,Two,Three'` , split this string on commas and then join the resulting list with a hyphen (–).

In [10]:
```
1 text = 'One,Two,Three'
2 text = text.split(',')
3 "-".join(text)
4
```

Out[10]:
```
'One-Two-Three'
```

Lists: Basic Operations & Slicing

List Indexing

Create a list `numbers = [10, 20, 30, 40, 50]` . How do you access the element `30` by its index?

In [11]:
```
numbers = [10, 20, 30, 40, 50]
print(numbers[2])
```

```
30
```

List Slicing

Given `numbers = [1, 2, 3, 4, 5, 6]` , slice the list to obtain only `[2, 3, 4]` .

In [12]:
```
numbers = [1, 2, 3, 4, 5,6]
numbers[1:4]
```

Out[12]:
```
[2, 3, 4]
```

Negative Index in Lists

Given `items = ['apple', 'banana', 'cherry', 'date']` , retrieve `'date'` using negative indexing.

In [13]:
```
items = ['apple', 'banana', 'cherry','date']
items[3]
```

Out[13]:
```
'date'
```

Appending to a List

Start with an empty list `my_list = []` . Use a list method to add the string `'Python'` to `my_list` .

In [14]:
```
my_list = []
my_list.append('Python')
my_list
```

Out[14]:
```
['Python']
```

Insert into a List

Given `colors = ['red', 'blue', 'green']` , insert `'yellow'` at the second position (index `1`) without overwriting existing elements.

In [15]:
```
colors = ['red', 'blue', 'green']
colors.insert(1, 'yellow')
colors
```

Out[15]:
```
['red', 'yellow', 'blue', 'green']
```

Extend a List

Given two lists `a = [1, 2]` and `b = [3, 4]`, combine them into a single list `[1, 2, 3, 4]` using a list method (not using `+` operator).

In [16]:
```
a=[1,2]
b=[3,4]
a.extend(b)
a
```

Out[16]:
```
[1, 2, 3, 4]
```

Removing from a List

Given `fruits = ['apple', 'banana', 'cherry']`, remove `'banana'` by name using a list method.

In [17]:
```
fruits = ['apple', 'banana', 'cherry']
fruits.remove('banana')
fruits
```

Out[17]:
```
['apple', 'cherry']
```

Pop from a List

From `numbers = [10, 20, 30, 40]`, remove the last item using a list method, and store it in a variable `removed_item`.

In [18]:
```
numbers = [10, 20, 30, 40]
removed_item = numbers.pop()
removed_item
```

Out[18]:
```
40
```

List Index Method

Given `cities = ['Paris', 'London', 'New York', 'Berlin']`, find the numeric index of `'New York'`.

In [19]:
```
cities = ['Paris','London','New York','Berlin']
cities.index('New York')
```

Out[19]: 2

Sorting a List

Given `ages = [25, 19, 42, 17]`, sort the list in ascending order using a list method.

In [20]:
```
ages = [25,19,42,17]
ages.sort()
ages
```

Out[20]: [17, 19, 25, 42]

Nested Lists

Indexing a Nested List

Given `nested = [['a', 'b'], ['c', 'd']]`, retrieve the element `'c'`.

In [21]:
```
nested = [['a','b'],['c','d']]
nested[1][0]
```

Out[21]: 'c'

Nested List Slicing

Given `matrix = [[1,2,3], [4,5,6], [7,8,9]]`, use slicing to get `[4,5,6]` (the second inner list).

In [22]:
```
matrix = [[1,2,3], [4,5,6], [7,8,9]]
matrix[1:2]
```

Out[22]:
```
[[4, 5, 6]]
```

Nested Access

From the same `matrix = [[1,2,3], [4,5,6], [7,8,9]]` , retrieve the element `8` directly (no loops).

In [23]:
```
matrix = [[1,2,3], [4,5,6], [7,8,9]]
matrix[2][1]
```

Out[23]:
```
8
```

Modifying a Nested List

Given `nested = [['apple', 'banana'], [10, 20]]` , replace the value `20` with `30` by direct indexing.

In [24]:
```
nested = [['apple', 'banana'], [10, 20]]
nested[1][1] = 30
nested
```

Out[24]:
```
[['apple', 'banana'], [10, 30]]
```

Slicing Elements of Nested Lists

Suppose you have `nested = [['hello','world'], ['python','rocks']]` . Use slicing on the string `'hello'` (inside the nested list) to get `'he'` .

In [25]:
```
nested = [['hello','world'], ['python','rocks']]
nested[0][0][:2]
```

Out[25]:
```
'he'
```

More String & List Methods

Concatenating Strings in a List

Given `words = ['Python', 'is', 'fun']`, combine them into a single string `'Pythonisfun'` without using the join method.

In [26]:
```
words = ['Python', 'is','fun']
for i in words:
    print(i,end='')
```

```
Pythonisfun
```

Index & Slice

From `quote = 'Simplicity is the soul of efficiency'`, slice out `'soul'` using only its indexes and slicing.

In [27]:
```
quote = 'Simplicity is the soul of efficiency'
quote[18:22]
```

Out[27]:
```
'soul'
```

Reversing a List with Slicing

Given `letters = ['A', 'B', 'C', 'D']`, reverse the entire list using slicing with a step.

In [28]:
```
letters = ['A','B','C','D']
letters[::-1]
```

Out[28]:
```
['D', 'C', 'B', 'A']
```

Replace a Slice

Given `nums = [1, 2, 3, 4, 5]`, replace the slice `[2, 3, 4]` with `['two', 'three', 'four']` using slice assignment.

In [29]:
```
nums = [1,2,3,4,5]
nums[1:4] = ['two','three','four']
nums
```

Out[29]:
```
[1, 'two', 'three', 'four', 5]
```

List count Method

Given `chars = ['x', 'y', 'x', 'z', 'x']` , use the appropriate list method to find how many times `'x'` appears.

In [30]:
```
chars = ['x','y','x','z','x']
chars.count('x')
```

Out[30]:
```
3
```

Logical / Boolean Expressions (No Loops)

Boolean AND

Without using loops, evaluate if `x = 5` is both greater than `3` and less than `10` .

In [31]:
```
x = 5
if x>3 and x<10:
    print('x is between 3 and 10')
```

```
x is between 3 and 10
```

Boolean OR

Given `brand = 'apple'` , check if `brand == 'apple' or brand == 'samsung'` .

In [32]:
```
brand = 'apple'
if brand == 'apple':
    print('Apple')
elif brand == 'samsung':
        print('Samsung')
```

```
Apple
```

Boolean NOT

Given `result = True` , flip it using `not` and store in `flipped_result` .

In [33]:
```
result = True
flipped_result = not result
flipped_result
```

Out[33]:
```
False
```

Comparisons in a Single Expression

Evaluate whether `5 < 10 == 10` . What does this return in Python (True/False) and why?

In [34]:
```
# the expression 5<10==10 is evaluated as (5<10) and (10==10) thus as it gives True and True which is Tru
5<10==10
```

Out[34]:
```
True
```

Multiple Conditions

Suppose `animal = 'cat'` and `color = 'black'` . Check if `animal` is `'cat'` and `color` is not `'white'` in a single expression.

In [35]:
```
animal = 'cat'
color = 'black'
if animal == 'cat' and color != 'white':
    print('Not a white cat')
```

```
Not a white cat
```

In [36]:
```
1 animal == 'cat' and color != 'white'
```

Out[36]:
```
True
```

Slightly Advanced Slicing & Methods (Hints Provided)

String Palindrome Check

How can you check if `txt = 'racecar'` is a palindrome using slicing?

In [37]:
```
1 txt = 'racecar'
2 rev = txt[::-1]
3 if txt == rev:
4     print('Palindrome')
```

```
Palindrome
```

Extracting Odd Indices

Given `nums = [0,1,2,3,4,5,6]` , slice out elements at odd indices.

In [38]:
```
1 nums = [0,1,2,3,4,5,6]
2 nums[1:6:2]
```

Out[38]:
```
[1, 3, 5]
```

Sorting a Nested List by an Inner Element

Given `pairs = [[2,'b'], [1,'a'], [3,'c']]` , sort them by the numeric element.

In [39]:
```
pairs = [[2,'b'], [1,'a'], [3,'c']]
pairs.sort(key=lambda x: x[0])
pairs

```

Out[39]:
```
[[1, 'a'], [2, 'b'], [3, 'c']]
```

Flattening a Nested List

If you have `nested_nums = [[1,2],[3,4]]` , flatten it into `[1,2,3,4]` .

In [40]:
```
nested_nums = [[1,2],[3,4]]
flat = [x for i in nested_nums for x in i]
flat
```

Out[40]:
```
[1, 2, 3, 4]
```

Slicing a List to Remove the First and Last Elements

Given `items = [100, 200, 300, 400, 500]` , slice to remove both the first and last elements.

In [41]:
```
items = [100, 200, 300,400,500]
items[1:4]
```

Out[41]:
```
[200, 300, 400]
```

In [ ]:
```

```